

[전공 심화 프로젝트] SafeWatch: 웹캠 기반 노인 안전 모니터링 서비스 사전 준비, 전체 계획 보고서

팀원: 김도현(FE/BE), 정선민(AI/영상처리)

1. 프로젝트 개요 (Project Overview)

본 프로젝트는 독거 고령자의 안전 사고(낙상)를 실시간으로 감지하고 보호자에게 즉시 알리는 웹 기반 모니터링 시스템이다. 별도의 착용 장치 없이 일반 웹캠만으로 동작하여 사용자의 거부감을 최소화하고, 낙상 이후의 무동작 상태를 감지하여 알림의 신뢰도를 높이는 것을 핵심 목표로 한다.

2. 최종 기술 스택 (Final Tech Stack)

AI 코딩 도구의 코드 생성 효율과 시스템의 실시간성을 고려하여 최적의 스택을 선정하였다.

2.1 개발 환경 및 프레임워크

- Frontend:** React(라이브러리)+typescript(언어) (빌드 Vite 기반), Axios, tailwindcss(css), zustand(전역상태)
- Main Backend:** Node.js (Express.js)
- AI Worker:** Python (FastAPI), OpenCV, PyTorch/TensorFlow
- Database:** PostgreSQL (Cloud/Local)
- ORM:** Prisma ORM (Type-safe 데이터 관리)

2.2 인증 및 통신

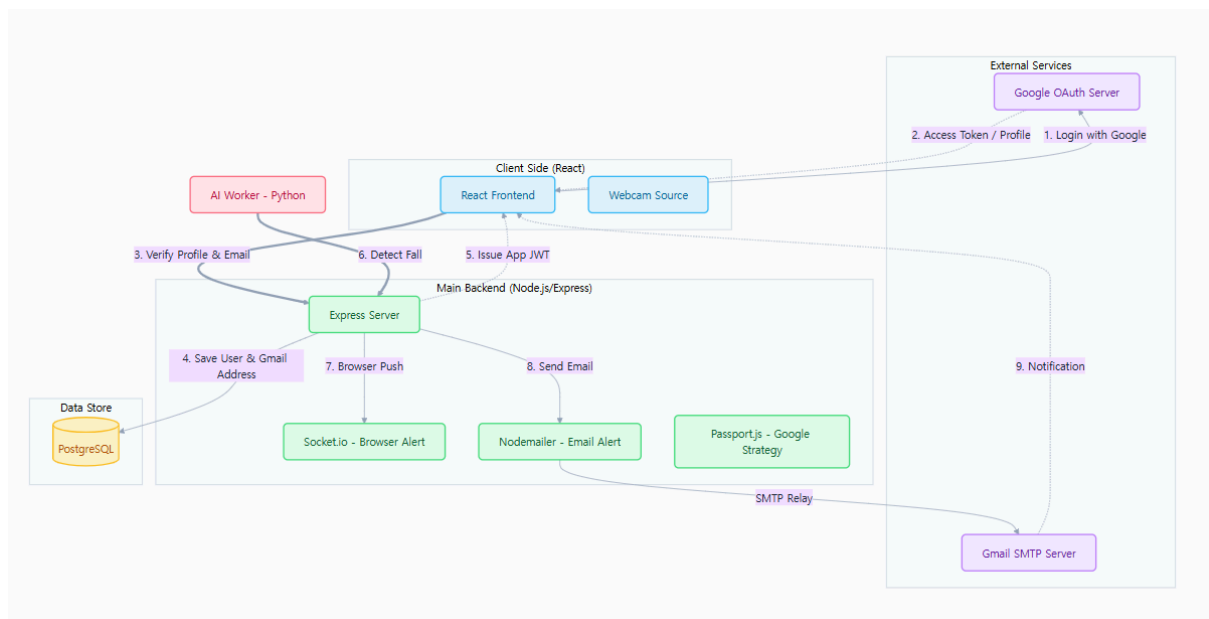
- Authentication:** Google OAuth 2.0 (보호자 이메일 연동 및 보안 강화)
- Real-time Alert:** Socket.io (브라우저 푸시 알림)
- Email Service:** Nodemailer (Gmail SMTP 기반 보조 알림)

3. 시스템 아키텍처 설계 (System Architecture)

본 시스템은 **3-Tier** 아키텍처를 기반으로 하되, 영상 분석 부하를 분산하기 위해 **AI Worker** 서버를 독립적으로 운영하는 구조를 채택하였다.

3.1 구성 요소별 역할

1. **Client (React):** 웹캠 스트리밍 및 모니터링 대시보드 제공. 낙상 발생 시 모달 알림 출력.
2. **Main Backend (Node.js):** 비즈니스 로직의 중심. 구글 로그인 처리, 이벤트 데이터 저장, 실시간 알림 트리거 및 중계.
3. **AI Worker (Python):** 실시간 영상 프레임을 받아 GRU 기반 모델로 낙상 여부 판단 후 메인 서버로 신호 전송.
4. **Database (PostgreSQL):** 사용자 정보, 피관찰자 설정, 낙상 이벤트 이력 저장.



4. 핵심 API 명세 설계 (API Specification)

AI 코딩 도구(Cursor 등) 활용 시 컨텍스트로 제공할 핵심 인터페이스이다.

기능 분류	Method	Endpoint	설명
인증	POST	/api/auth/google	구글 ID 토큰 검증 및 JWT 발급
이벤트	POST	/api/events/detect	AI Worker의 낙상 감지 신호 수신 및 저장
이력 조회	GET	/api/events	특정 사용자의 최근 낙상 이력 목록 조회
사용자	GET	/api/user/profile	보호자 정보 및 알림 수신 설정 조회
알림 테스트	POST	/api/test/email	가입된 구글 이메일로 테스트 알림 발송

5. 데이터베이스 스키마 설계 (DB Schema)

Prisma ORM을 활용하여 데이터 일관성을 유지한다.

- **User Table:** id, email (Google), name, provider_id, created_at
- **FallEvent Table:** id, user_id (FK), timestamp, confidence, status (감지/확인됨), image_url (캡처본)

6. 초기 개발 환경 구축 현황 (Implementation Status)

현재 원활한 기능 구현을 위한 기초 환경 구축을 완료하였다.

1. **Backend Base:** Node.js Express 환경에 Prisma를 연동하여 PostgreSQL 서버와의 통신 확인.
2. **Auth Setup:** 미구현
3. **Frontend Base:** React(Vite) 프로젝트 생성, 디자인 생성 완료
4. **AI Workspace:** Python 가상환경을 구성하고, OpenCV·YOLO 기반 영상 처리 및 객체 탐지 개발 환경을 구축중. 웹캠 입력 테스트 코드 작성 완료. 기본 추론 파이프라인 및 낙상 판단 로직 연동을 위한 AI 서버 베이스를 추후 구축 예정.
5. **CI/CD Setup:** GitHub Actions 기반 자동 테스트 및 빌드 환경 구성. 코드 변경 시 백엔드 API 동작, 프론트엔드 빌드, AI 서버 실행 가능 여부를 자동으로 검증.

7. AI 코딩 도구 기반 개발 전략 (AI-Driven Development Strategy)

제공된 가이드라인에 따라 '이해 중심의 반복 개발'을 수행한다.

1. 기능 단위 분리: 전체 시스템을 '구글 로그인', '낙상 알림 전송', '이벤트 조회' 등의 작은 단위로 쪼개어 개발한다.
2. 컨텍스트 제공: AI에게 요청 시 위에서 정의한 '프로젝트 목적'과 'API 명세'를 항상 상단에 배치하여 일관된 코드를 생성하게 한다.
3. 검증 후 통합: AI가 생성한 코드를 줄 단위로 검토하여 의도와 맞는지 확인한 후 메인 프로젝트에 병합한다.

8. 향후 일정 (Future Plan)

- **1단계 (중간 이후):** 구글 로그인 기능 완성 및 DB 사용자 연동.
- **2단계:** AI Worker에서 보낸 신호를 실시간 브라우저 알림으로 띄우는 핵심 흐름(Critical Path) 구현.
- **3단계:** 이메일 알림 및 대시보드 이력 조회 기능 완성.

